C# Methods, Fields, Properties, Classes, and Interfaces

1. Methods and Parameters

A **method** represents an operation or behavior. It groups statements under a name so you can call the operation whenever you need it.

A method may receive zero or more **parameters** and may either return a value or return nothing (void).

```
public int AddNumbers(int number1, int number2)
{
    return number1 + number2;
}
```

The method has two input parameters:

`number1`

`number2`

and returns one **`int`**.

A call such as:

`int result = AddNumbers(2, 3);`

passes 2 to number1 and 3 to number2.

1.1 Arguments vs. Parameters

These terms are closely related but have different meanings.

`public int AddNumbers(int number1, int number2)`

number1 and number2 are **parameters** because they appear in the method declaration.

`AddNumbers(10, 5);`

`10` and `5` are **arguments** because they are the actual values supplied when the method is called.

Tip — Parameter vs. Argument

Parameter: variable declared by the method.
Argument: value supplied by the caller.

---

2. Passing Parameters by Value

C# parameters are **passed by value by default**. This means the method receives a copy of the argument's value.

Consider:

```
int number = 20;
AddFive(number);

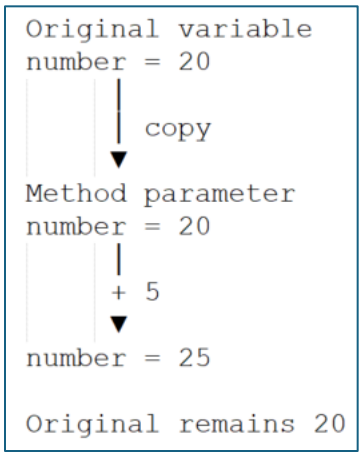
Console.WriteLine(number);

void AddFive(int number)
{
    number += 5;
}
```

The output remains:

20

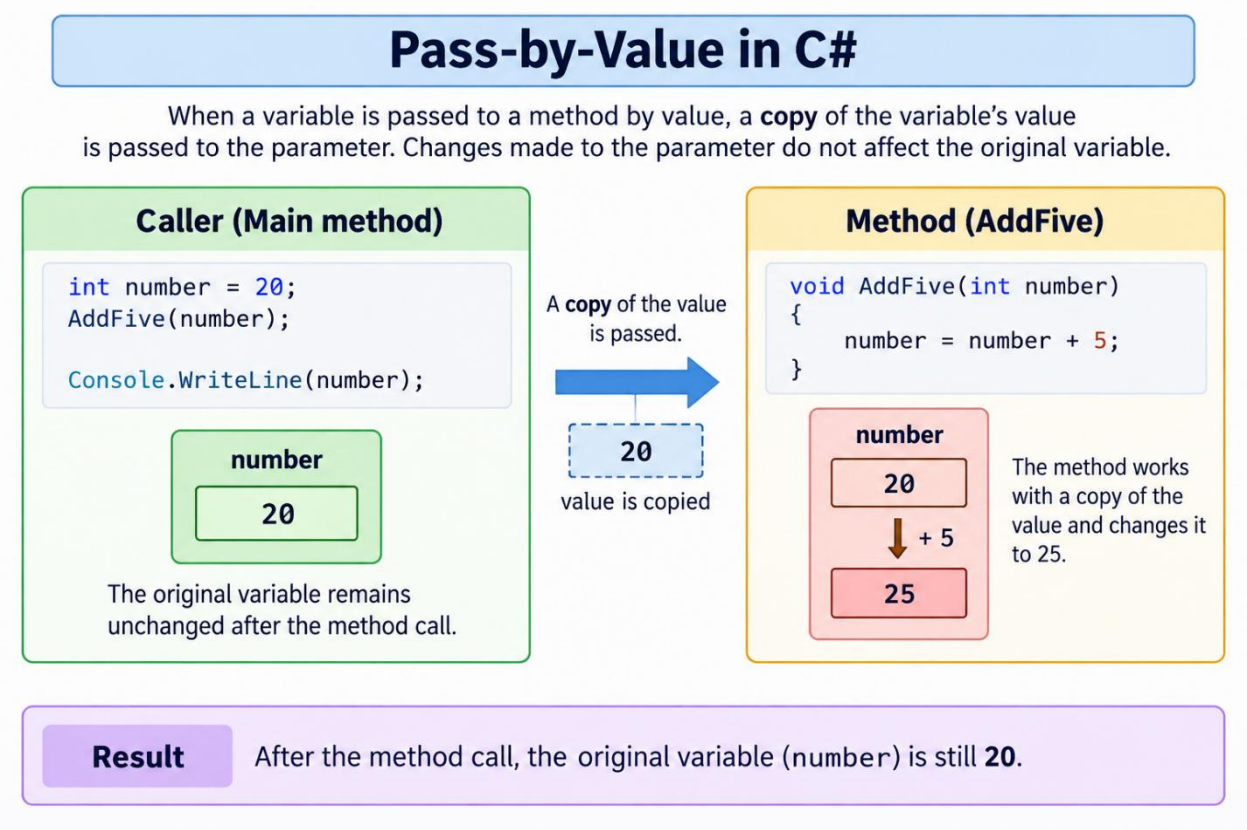
Conceptually:



The parameter and original variable are separate variables.

Important

"Passed by value" means the **value of the variable is copied into the parameter**. It does not simply mean "value types are passed by value." The distinction becomes especially important when working with reference types.



---

3. Passing Parameters by Reference

Sometimes a method needs to work directly with the caller's variable. C# provides **`ref`**, **`out`**, and **`in`** for different forms of by-reference parameter passing.

3.1 `ref` Parameters

`ref` allows the method to access and modify the caller's variable directly.

```
int number = 20;

AddFive(ref number);

Console.WriteLine(number);

void AddFive(ref int number)
{
    number += 5;
}
```

Output:

```
25
```

Notice that ref appears in **both places**:

```
AddFive(ref number);
```

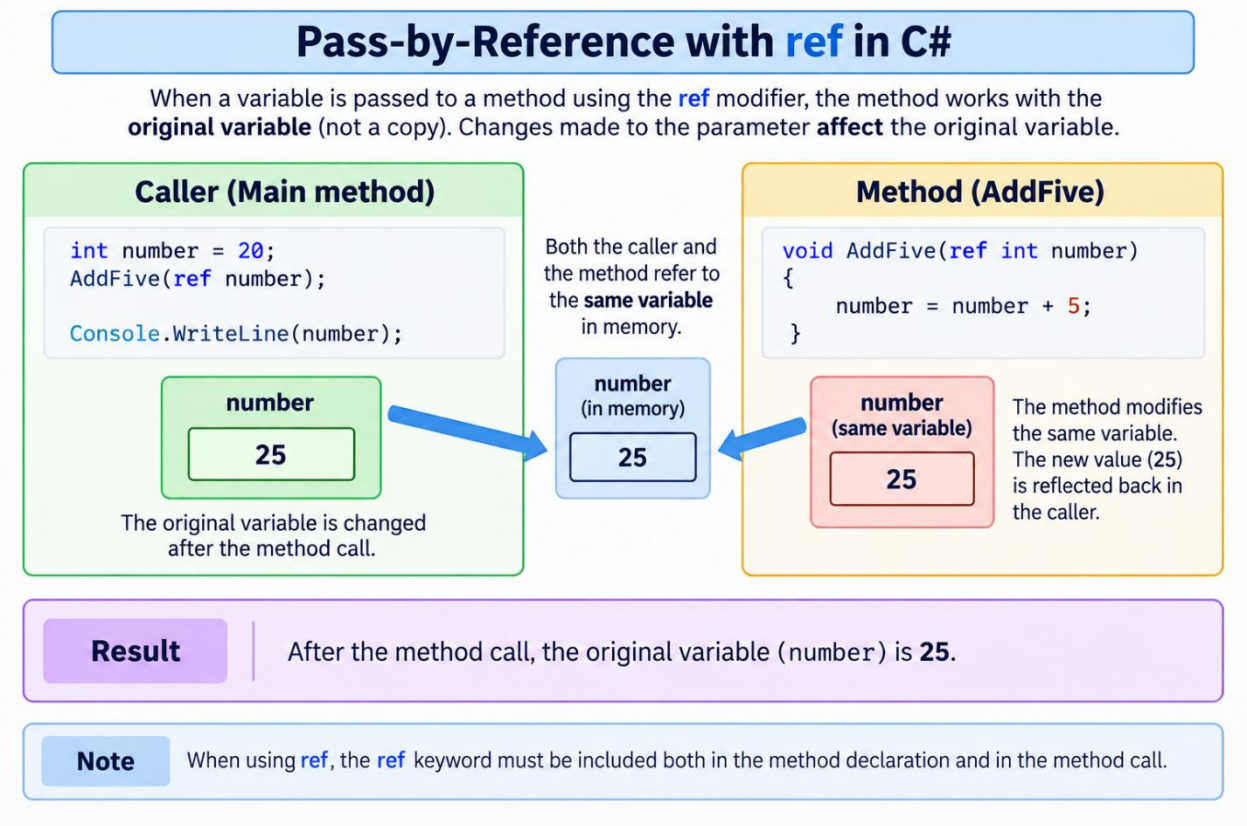
and:

```
void AddFive(ref int number)
```

Conceptually:

```
Caller
number = 20
    ▲
    | same variable
    |
AddFive(ref number)
    |
    └── changes it to 25
```

Unlike the earlier example, the method modifies the original variable.



3.2 `out` Parameters

`out` also passes a variable by reference, but its purpose is different: the method uses the parameter to provide an output value.

A method can therefore produce multiple results.

```
int addedValue;
int subtractedValue;

DoMath(10, 5, out addedValue, out subtractedValue);

void DoMath(
    int number1,
    int number2,
    out int addedValue,
    out int subtractedValue)
{
    addedValue = number1 + number2;
    subtractedValue = number1 - number2;
}
```

After the call:

`addedValue` `= 15`

`subtractedValue` `= 5`

An **`out`** parameter must be assigned by the method before the method returns.

Note — ref vs. out

`ref` is appropriate when the caller already has a value that the method may read and modify.

`out` is designed for a value that the method is responsible for producing.

3.3 `in` Parameters

An **`in`** parameter is also passed by reference, but the method is prevented from assigning a new value to the parameter.

```
void Display(in string message)
{
    Console.WriteLine(message);
}
```

Think of the three modifiers as:

`ref` → read + potentially modify

`out` → method produces the value

`in` → read without assigning to parameter

---

4. Optional Parameters

A parameter can have a **default value**, making the argument optional when the method is called.

```
public int AddNumbers(
    int number1,
    int number2,
    int number3 = 0)
{
    return number1 + number2 + number3;
}
```

Both calls are valid:

`AddNumbers(2, 3);`

`AddNumbers(2, 3, 10);`

In the first call, **`number3`** receives its default value.

Pitfall — Parameter Order

Required parameters should appear before optional parameters. Once optional parameters begin, subsequent parameters normally also need default values unless another language feature changes how the call is expressed.

---

5. Named Arguments

Normally, arguments correspond to parameters by their position.

`PrintUserDetails(1, "John");`

Named arguments make the mapping explicit:

```
PrintUserDetails(
    name: "John",
    userId: 1);
```

This is particularly useful when a method has several parameters or optional parameters. The course examples specifically use named arguments to supply optional values without supplying every preceding optional argument.

For example:

```
PrintUserDetails(
    addressLines: new List<string>(),
    name: "Jane",
    userId: 2);
```

Tip

Named arguments can improve readability when several arguments have similar types. A call containing four integers is difficult to interpret without seeing the method declaration; named arguments can make the purpose of each value explicit.

---

6. Variable Numbers of Arguments with `params`

Sometimes the number of arguments is not known in advance.

The **`params`** keyword allows a method to receive a variable number of arguments as a **single-dimensional array**. Only one **`params`** parameter is permitted, and no parameters can appear after it.

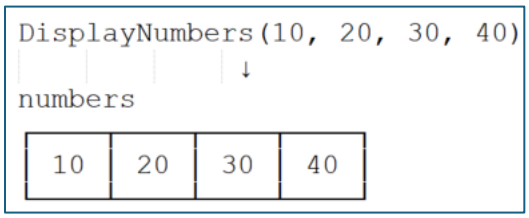
```
static void DisplayNumbers(params int[] numbers)
{
    foreach (int number in numbers)
    {
        Console.WriteLine(number);
    }
}
```

It can then be called with different numbers of arguments:

```
DisplayNumbers(1, 2);

DisplayNumbers(1, 2, 3, 4);
```

Conceptually:



---

7. Fields: Storing Object State

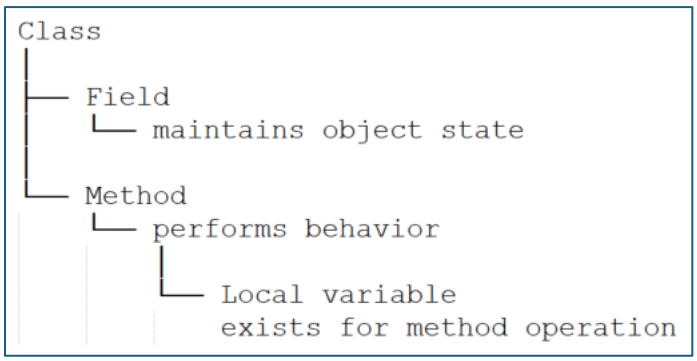
Methods represent **behavior**. Objects also need **state**.

A **field** is a variable declared directly inside a class or struct. Fields are members of their containing type and are appropriate for data that must survive beyond a single method invocation. Data needed only temporarily by one method should normally remain a local variable.

```
public class Animal
{
    0 references
    private string _name;
}
```

Here **`_name`** is a field.

Compare:



7.1 Instance Fields

An **instance field** belongs to an individual object.

Suppose two **`Animal`** objects exist:

`Animal object #1`

`_name = "Fido"`

Animal object #2

_name = "Garfield"

Changing the first object's **_name** does not change the second object's **_name**. Each instance has its own state.

---

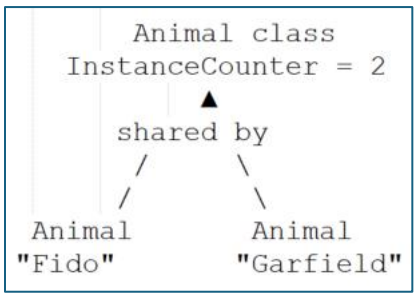
8. Static Fields

A **static field** belongs to the type itself rather than to one particular object. Consequently, it is shared across instances.

A typical scenario is counting created objects:

```
public class Animal
{
    0 references
    private string _name;
    0 references
    public static int InstanceCounter;
}
```

Conceptually:



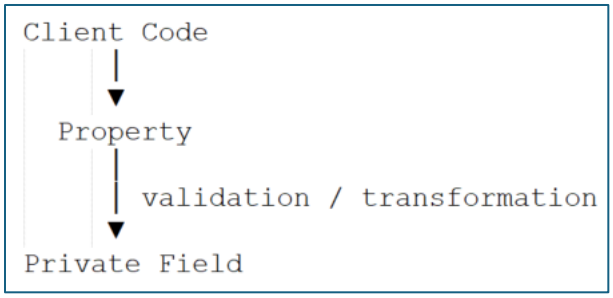
The individual **_name** values belong to objects, while the static counter belongs to the class.

---

9. Encapsulation: Do Not Expose Fields Directly

Fields are generally implementation details. Data intended for client code should normally be exposed through controlled members such as properties. This creates a boundary where validation or other logic can be applied.

Instead of designing the class around publicly writable fields, think in terms of:



This leads directly to **properties**.

---

10. C# Properties

A **property** provides controlled access to data while presenting convenient field-like syntax to client code. Properties use accessors such as get and set.

For example:

```
public string Name { get; set; }
```

Client code can use it naturally:

```
animal.Name = "Fido";

Console.WriteLine(animal.Name);
```

The syntax resembles accessing a field, but Name is a property.

Accessors

`get` retrieves a value.

`set` assigns a value.

Within a **`set`** accessor, the keyword:

`value`

represents the value being assigned.

---

11. Auto-Implemented Properties

When no additional logic is necessary, an auto-implemented property provides concise syntax:

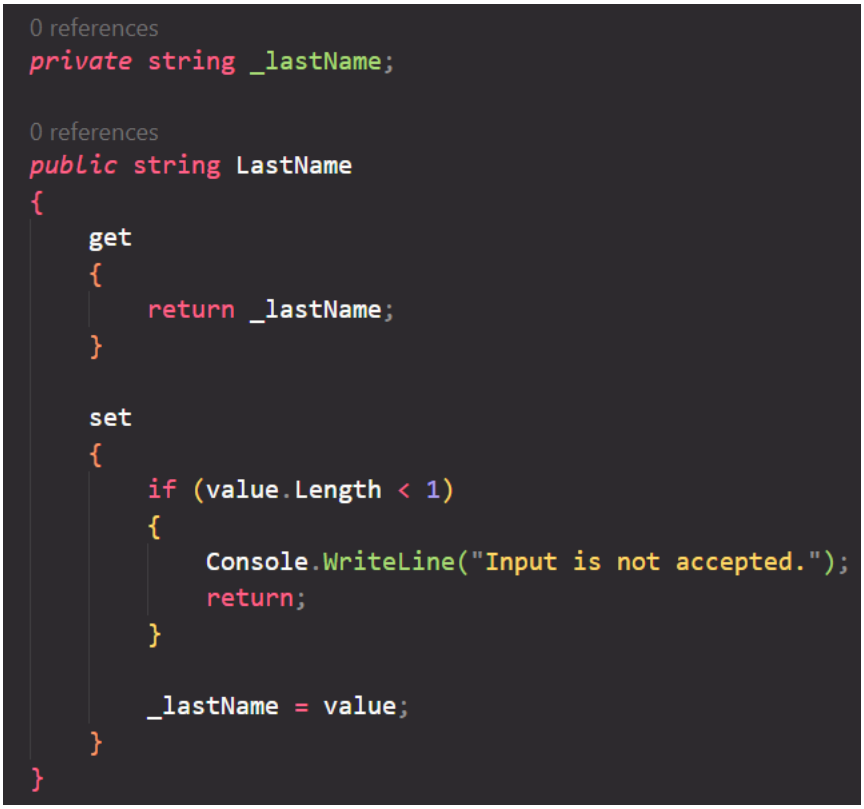
```
public string Name { get; set; }
```

Use this when simply storing and retrieving a value is sufficient.

---

12. Properties with Backing Fields

When additional control is necessary, use a **private backing field**.



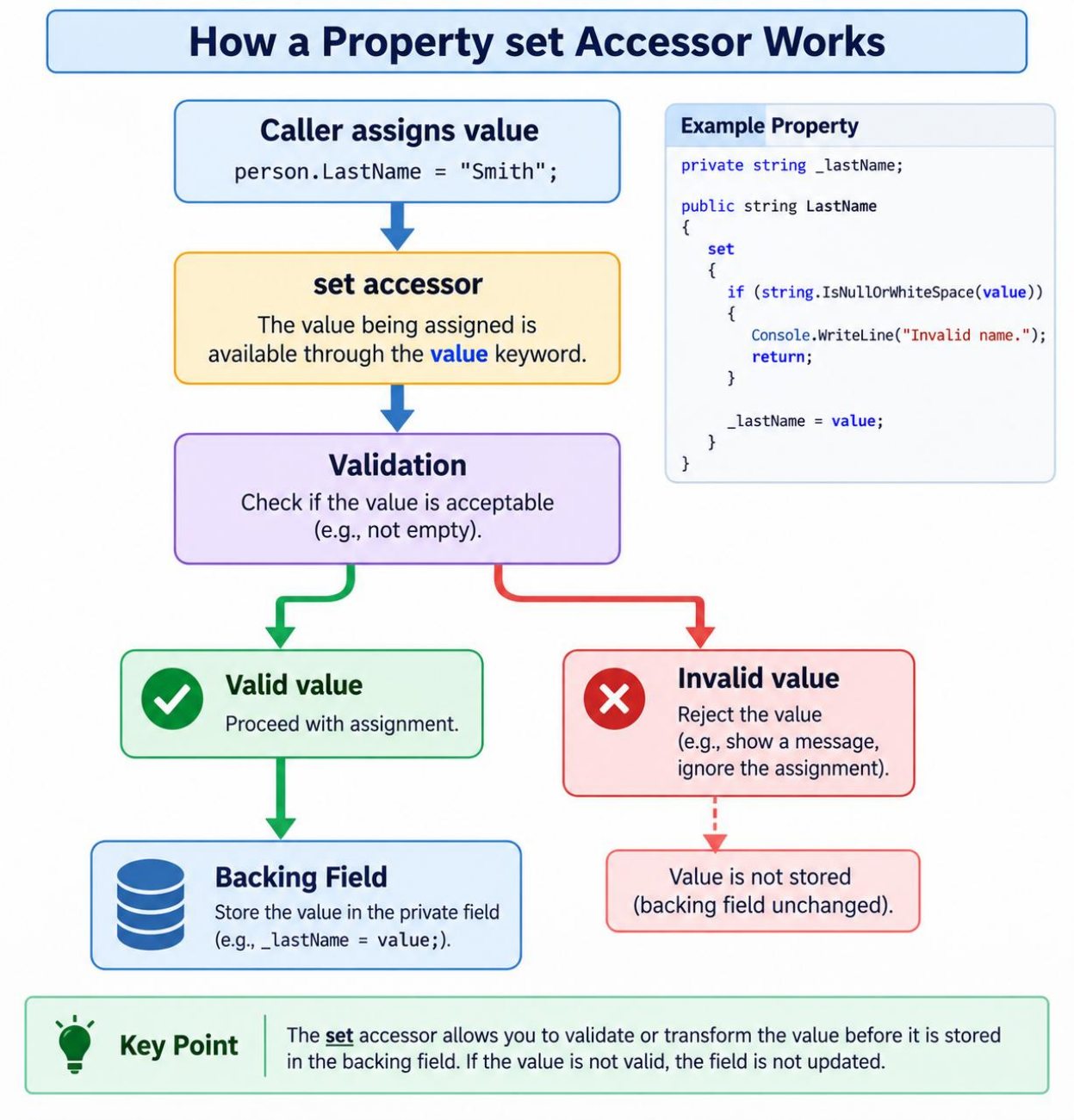
```
0 references
private string _lastName;

0 references
public string LastName
{
    get
    {
        return _lastName;
    }

    set
    {
        if (value.Length < 1)
        {
            Console.WriteLine("Input is not accepted.");
            return;
        }

        _lastName = value;
    }
}
```

The important sequence is:



The property therefore becomes a layer of abstraction over the underlying field and can validate, convert, or compute data before storing or returning it.

Tip — Choosing the Property Style

Use an **auto-property** when no special behavior is required.

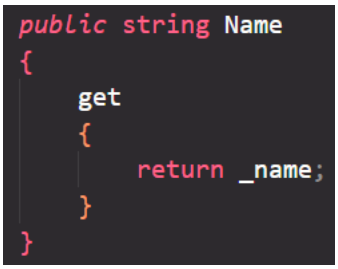
Use a **backing field** when getting or setting the value requires validation, conversion, computation, or other behavior.

---

13. Read-Only and Write-Only Properties

A property does not necessarily need both accessors.

A property containing only get provides read access:



```
public string Name
{
    get
    {
        return _name;
    }
}
```

A property containing only **`set`** provides write access.

The examples in this section demonstrate get-only, set-only, and combined get/set properties.

---

14. Expression-Bodied Properties

When a property consists of a simple expression, C# allows a more concise form.

```
public string Name =>
    $"{_firstName} {_lastName}";
```

This is a read-only computed property.

If:

```
_firstName = "Jane"
```

```
_lastName  = "Smith"
```

then:

```
person.Name
```

produces:

```
Jane Smith
```

Individual accessors can also use expression-bodied syntax:

```
public string Name
{
    get => _name;
    set => _name = value;
}
```

The expression-bodied forms covered here provide shorter syntax without changing the fundamental role of the property.

---

15. Classes and Objects

We can now combine the concepts.

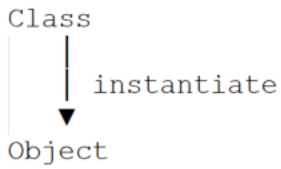
A **class** defines a type. Its members describe the state and behavior associated with that type.

Typical members include:



These are collectively known as **class members**.

A class itself is not an individual object.



For example:

```
Customer customer = new Customer();
```

`Customer` describes the type; **`customer`** refers to a concrete instance.

---

16. Classes Are Reference Types

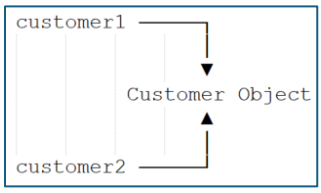
C# classes are **reference types**. A variable of a class type holds a reference to an object rather than containing the complete object directly.

Consider:

```
Customer customer1 = new Customer();
Customer customer2 = customer1;
```

This does **not** create a second independent Customer.

Conceptually:

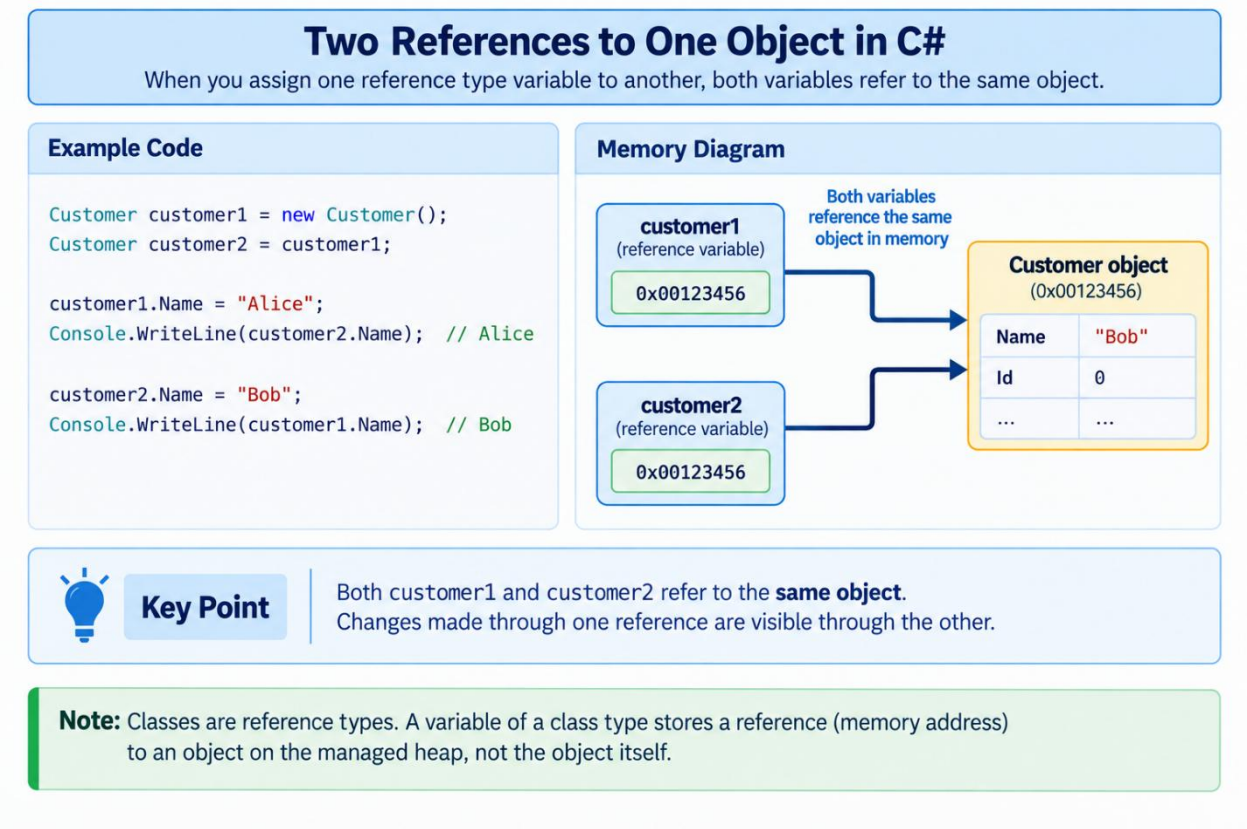


Both variables reference the same object.

The object itself is allocated on the managed heap, and the CLR provides automatic memory management through **garbage collection**.

Pitfall — Reference Assignment

Assigning one class variable to another copies the **reference**, not the complete object. Changes made through one reference can therefore be visible through the other.



17. Access Modifiers

Access modifiers determine where a type or member can be accessed.

Modifier	General accessibility
`public`	Accessible from other code
`private`	Only within the declaring class/struct
`protected`	Declaring type and derived classes
`internal`	Same assembly
`protected internal`	Same assembly **or** derived classes in another assembly
`private protected`	Derived types within the same assembly

The detailed accessibility rules are part of the class design material.

A common encapsulation pattern is:

```
public class Product
{
    2 references
    private decimal _price;

    0 references
    public decimal Price
    {
        get => _price;
        set => _price = value;
    }
}
```

The class is publicly accessible, while its internal state remains private and is exposed through a property.

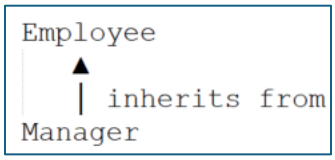
---

18. Inheritance

Inheritance allows a new class to derive data and behavior from another class.

```
public class Manager : Employee {
}
```

Here:

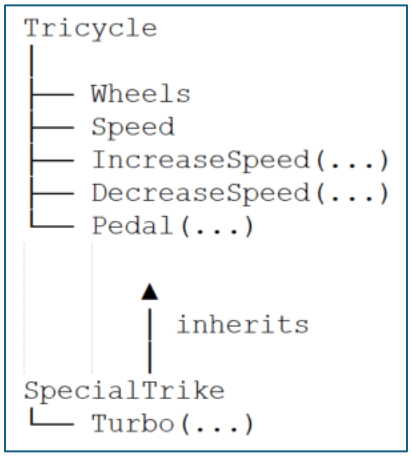


`Employee` is the **base class** and **`Manager`** is the **derived class**.

A class can directly inherit from only **one base class**, although inheritance can form a chain. Constructors are not inherited.

Practical Exercise — Tricycle Hierarchy

Create:



Implement the appropriate backing fields and access modifiers, then test whether the derived class can access the intended inherited members.

This exercise reinforces fields, properties, methods, parameters, accessibility, and inheritance without introducing additional concepts.

---

19. Abstract Classes

An **abstract class** provides a base design that is intended to be completed by derived classes.

An abstract class:

- cannot be instantiated directly;
- may contain implemented members;
- may contain abstract members without implementations.

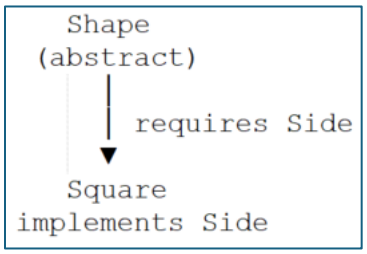
An abstract method declares what operation must exist but leaves its implementation to a derived class.

```
0 references
abstract class Shape
{
    0 references
    public abstract int Side { get; set; }
}
```

A concrete derived class must implement the abstract member:

```
0 references
class Square : Shape
{
    0 references
    public override int Side { get; set; }
}
```

Conceptually:



Important

`abstract` describes an incomplete member or type intended for inheritance.

`override` supplies or replaces inherited virtual/abstract behavior in the derived class.

Practical Exercise

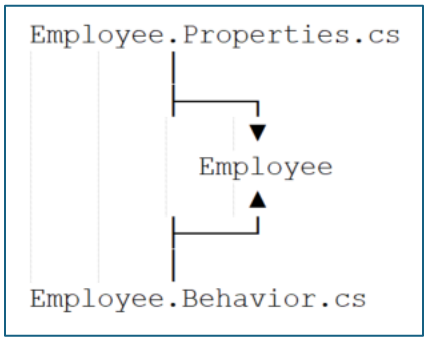
Create a **`Triangle`** class derived from **`Shape`**. Override the required abstract properties, add appropriate constructors, create an instance in the program, and demonstrate its behavior.

---

20. Partial Classes

The **`partial`** keyword allows one class definition to be split across multiple source files.

For example:



Each part uses:

`partial class Employee`

The compiler combines the parts into one type.

All parts must belong to the same assembly and namespace, use the **`partial`** keyword, and have compatible accessibility. Members declared in one part are available to the others.

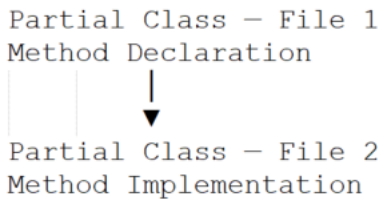
This allows a large type to be organized across multiple **`.cs`** files while still representing one class.

---

21. Partial Methods

Partial types can also contain **partial methods**, separating a method declaration from its implementation.

The deck's partial-method model separates:



The material specifies partial methods as **`void`**, implicitly private, non-virtual, optionally static or generic, and allowing in or **`ref`** but not **`out`** parameters.

---

22. Interfaces

An **interface** defines a **contract**.

A class or struct implementing that interface agrees to provide the required behavior. Interfaces can declare methods, properties, indexers, and events and do not contain instance state such as instance fields.

For example:

```
0 references
interface ISampleInterface
{
    0 references
    void FirstMethod();
    0 references
    void SecondMethod();
}
```

A class can implement the contract:

```
0 references
class ImplementationClass : ISampleInterface
{
    0 references
    public void FirstMethod()
    {
        Console.WriteLine("Inside FirstMethod");
    }

    0 references
    public void SecondMethod()
    {
        Console.WriteLine("Inside SecondMethod");
    }
}
```

The relationship is:

```
ISampleInterface
   "You must provide
    these operations."
          |
          ▼
ImplementationClass
   provides implementations
```

provides implementations

Key distinction

A class describes an implementation and can provide state and behavior.

An interface primarily establishes a contract that implementing types must satisfy.

---

23. Explicit Interface Implementation

An interface member can be implemented **explicitly**.

```
void ISampleInterface.SecondMethod()
{
    Console.WriteLine("Inside SecondMethod");
}
```

An explicitly implemented member is not called directly through a variable whose compile-time type is the implementing class.

Instead:

```
ISampleInterface obj =
    new ImplementationClass();

obj.SecondMethod();
```

The distinction demonstrated by the examples is:

```
ImplementationClass reference
        │
        └── normally accessible class members

ISampleInterface reference
        │
        └── interface contract members,
            including explicit implementations
```

The interface examples specifically demonstrate this difference between class-typed and interface-typed references.

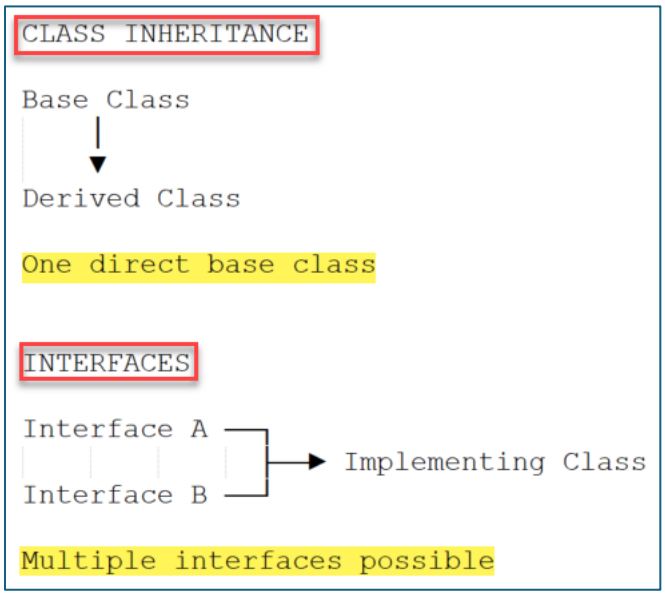
---

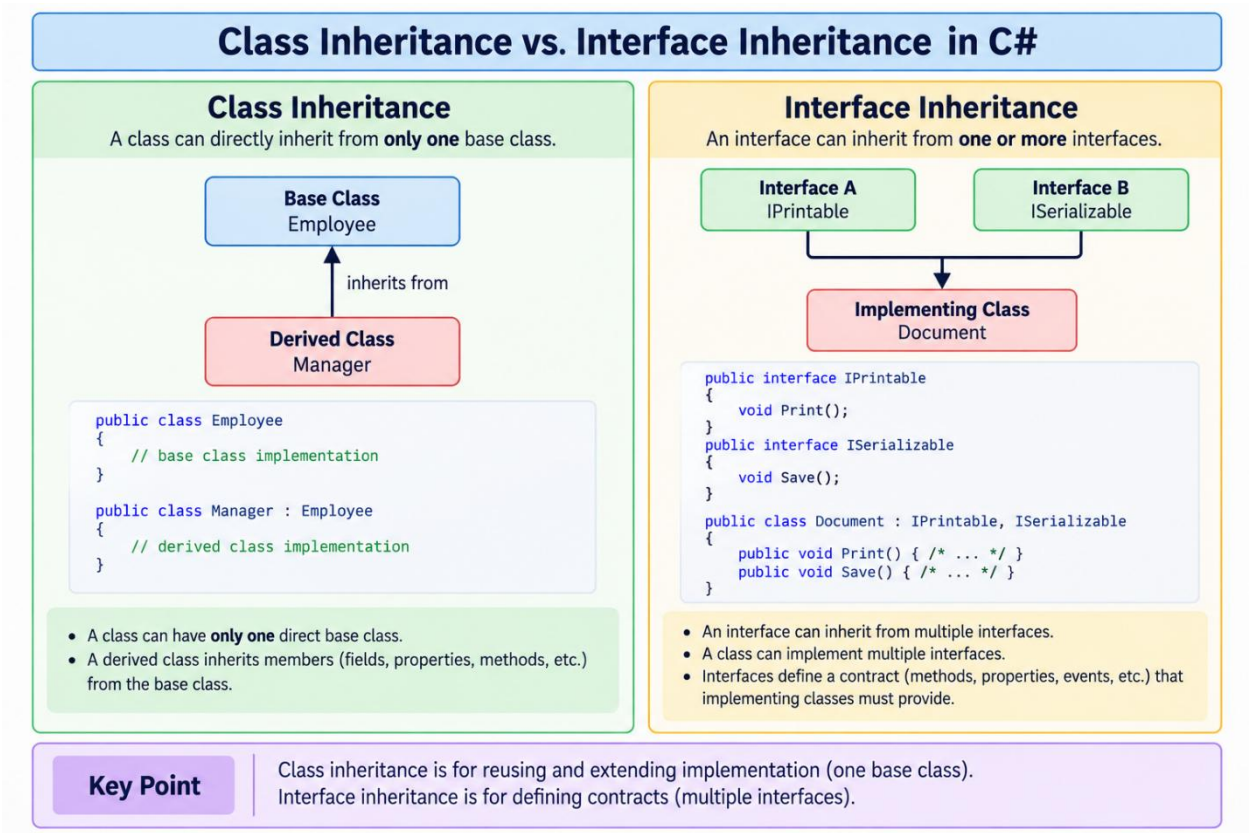
24. Interface Inheritance and Default Implementations

An interface can inherit from **one or more interfaces**. A class, by contrast, directly inherits from only one base class.

The material also introduces the C# 8 capability for interfaces to define default member implementations and static members.

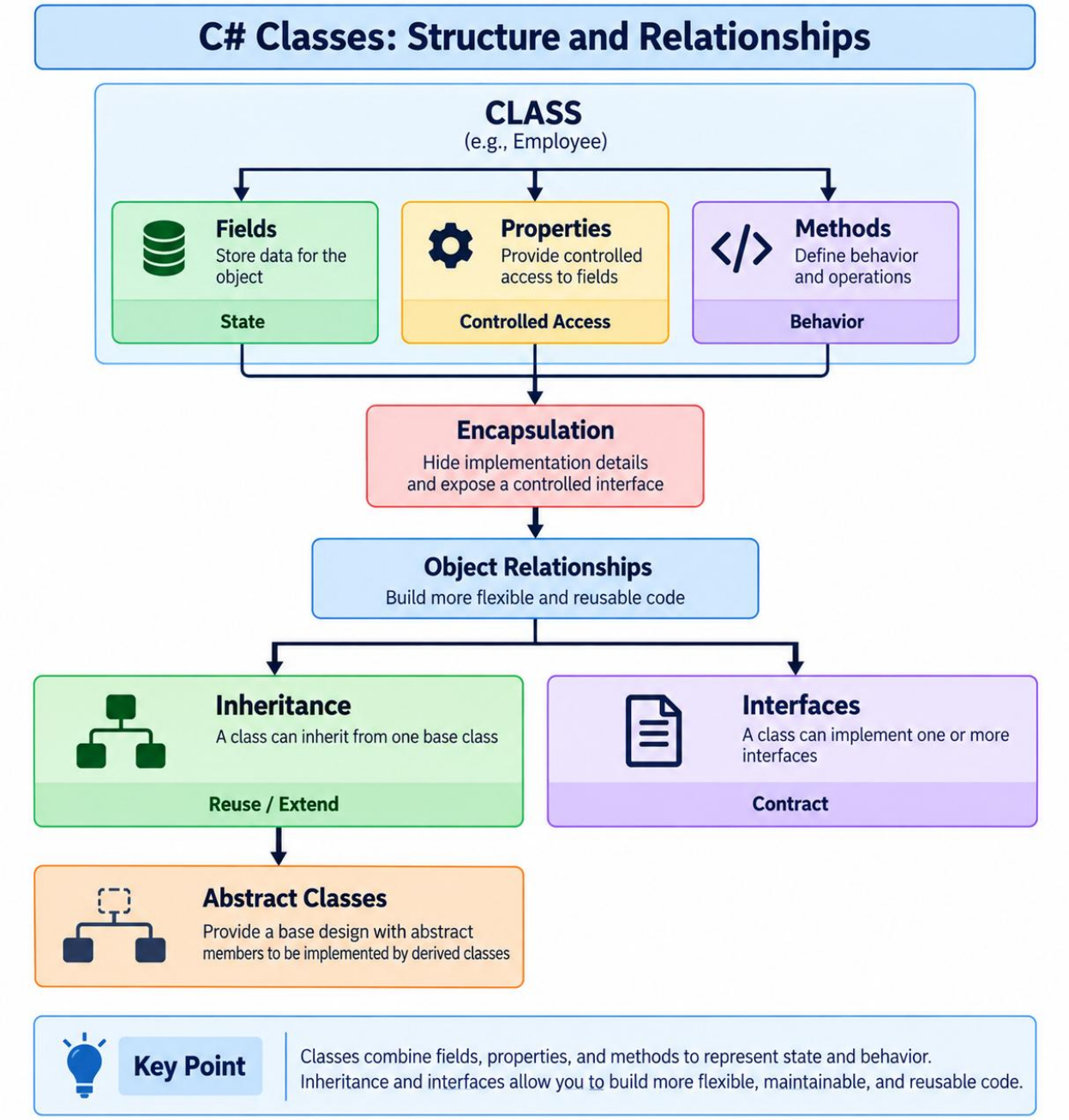
At this stage, the key conceptual distinction is:



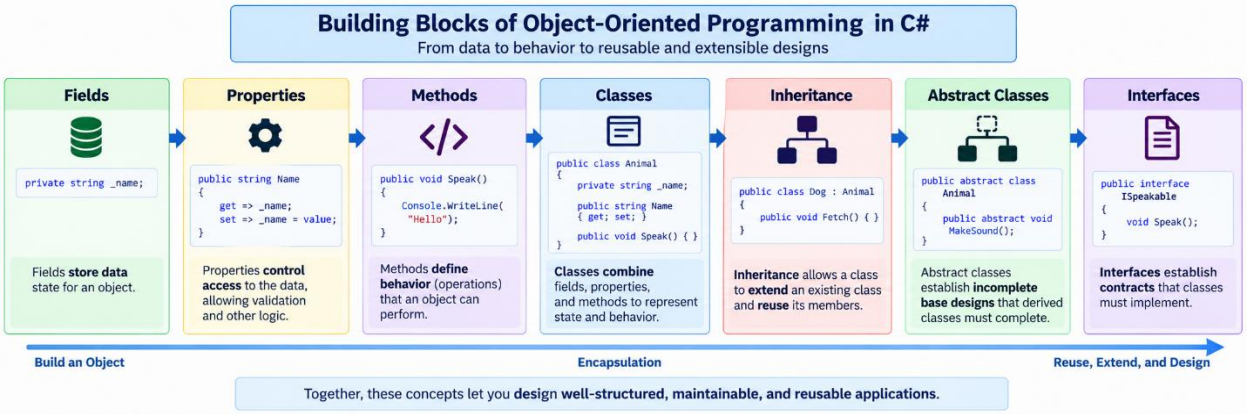


25. Putting the Concepts Together

These concepts form a coherent object-oriented model:



A practical way to think about the entire module is:


Building Blocks of Object-Oriented Programming in C#
From data to behavior to reusable and extensible designs
Fields
private string _name;
Fields store data state for an object.
Properties
public string Name
{
get => _name;
set => _name = value;
}
Properties control access to the data, allowing validation and other logic.
Methods
public void Speak()
{
Console.WriteLine(
"Hello");
}
Methods define behavior (operations) that an object can perform.
Classes
public class Animal
{
private string _name;
public string Name
{ get; set; }
public void Speak() { }
}
Classes combine fields, properties, and methods to represent state and behavior.
Inheritance
public class Dog : Animal
{
public void Fetch() { }
}
Inheritance allows a class to extend an existing class and reuse its members.
Abstract Classes
public abstract class
Animal
{
public abstract void
MakeSound();
}
Abstract classes establish incomplete base designs that derived classes must complete.
Interfaces
public interface
ISpeakable
{
void Speak();
}
Interfaces establish contracts that classes must implement.
Build an Object
Encapsulation
Reuse, Extend, and Design
Together, these concepts let you design well-structured, maintainable, and reusable applications.